

GitHub Link:

<https://github.com/Ayushgautam2007/CI-CD-Project.git>

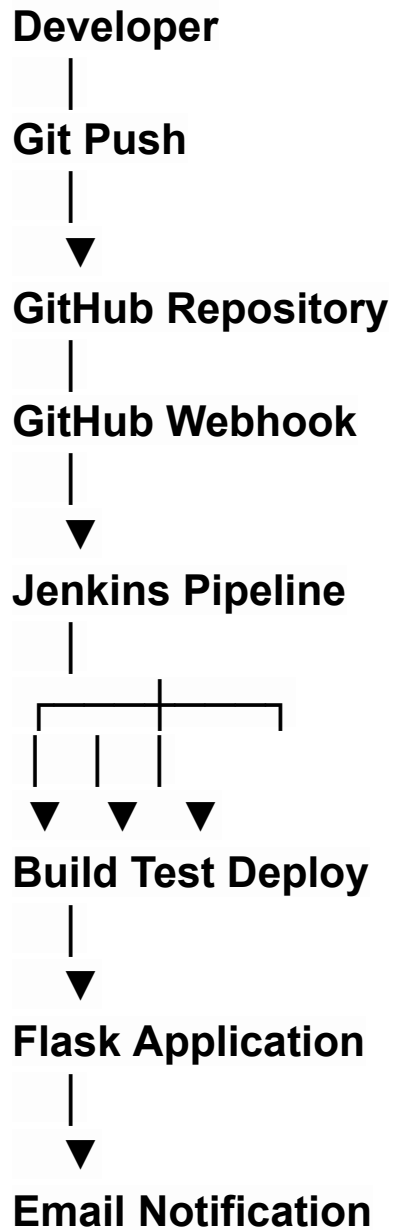
Project Objective

The objective of this project is to automate the build, testing, and deployment of a Flask web application using Jenkins and GitHub Actions. The project demonstrates Continuous Integration (CI) and Continuous Deployment (CD) by automatically installing dependencies, running unit tests, deploying the application, and sending notifications after every code change.

Technology Stack

- AWS EC2 (Amazon Linux 2023)
- Jenkins
- GitHub
- GitHub Actions
- Python 3.9
- Flask
- Pytest
- Git
- Gmail SMTP

1. Project Architecture - Jenkins CI/CD Pipeline:



1. Jenkins CI/CD Pipeline for Flask Application

This section demonstrates the implementation of a Jenkins-based CI/CD pipeline for a Flask web application. The pipeline automates dependency installation, testing, deployment, and email notifications whenever code is pushed to the GitHub repository.

Prerequisites

- AWS EC2 Instance
- Internet Connectivity
- Java 21
- Python 3.9
- pip
- Git
- GitHub Account
- Jenkins Installed

1. Configure Jenkins and Python libraries in EC2-Instance:
 - A. Configure Java-21 and Jenkins:

Command Used: `yum install java-21-amazon-corretto -y`

```
[root@ip-172-31-0-147 ~]# java --version
openjdk 21.0.11 2026-04-21 LTS
OpenJDK Runtime Environment Corretto-21.0.11.10.1 (build 21.0.11+10-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.11.10.1 (build 21.0.11+10-LTS, mixed mode, sharing)
[root@ip-172-31-0-147 ~]#
```

Purpose

Java is required because Jenkins runs as a Java application.

Configure Jenkins:

1. Add Jenkins Repo using command:

wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo

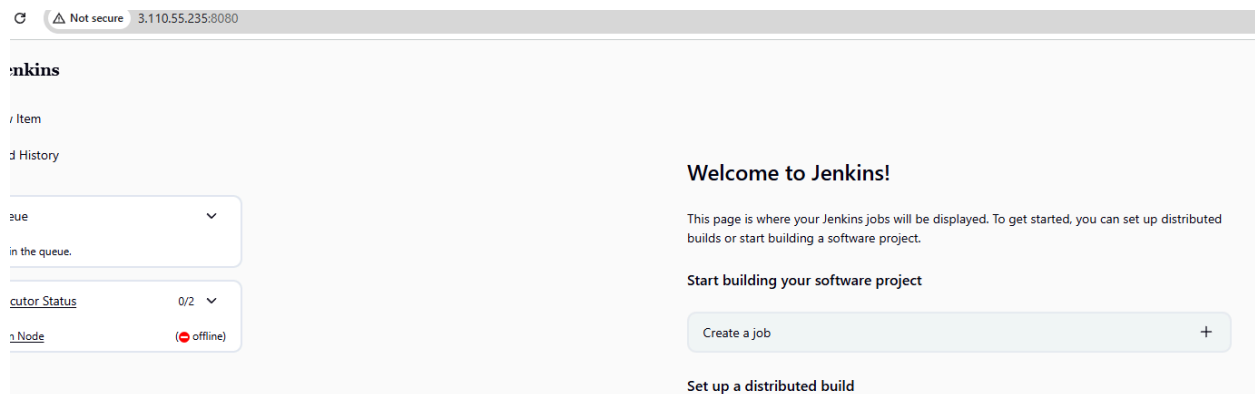
```
[root@ip-172-31-0-147 ~]# wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/redhat-stable/jenkins.repo
--2026-06-29 04:47:54-- https://pkg.jenkins.io/redhat-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 151.101.210.133, 2a04:4e42:31::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)[151.101.210.133]:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://pkg.jenkins.io/rpm-stable/jenkins.repo [following]
--2026-06-29 04:47:54-- https://pkg.jenkins.io/rpm-stable/jenkins.repo
Reusing existing connection to pkg.jenkins.io:443.
HTTP request sent, awaiting response... 200 OK
Length: 267 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'

/etc/yum.repos.d/jenkins.repo 100%[=====>] 267 --.-KB/s in 0s

2026-06-29 04:47:54 (12.9 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [267/267]

[root@ip-172-31-0-147 ~]#
```

2. Install Jenkins:



B. Configure python dependencies on server:

Using Command: yum install python3 python3-pip -y

```
[root@ip-172-31-0-147 ~]# python3 --version
Python 3.9.25
[root@ip-172-31-0-147 ~]# pip3 --version
pip 21.3.1 from /usr/lib/python3.9/site-packages/pip (python 3.9)
[root@ip-172-31-0-147 ~]#
```

2. Install git and required python packages for Flask:

Command Used: yum install git -y

```
[root@ip-172-31-0-147 ~]# git --version
git version 2.50.1
[root@ip-172-31-0-147 ~]#
```

Command used for python packages: pip3 install flask pytest gunicorn

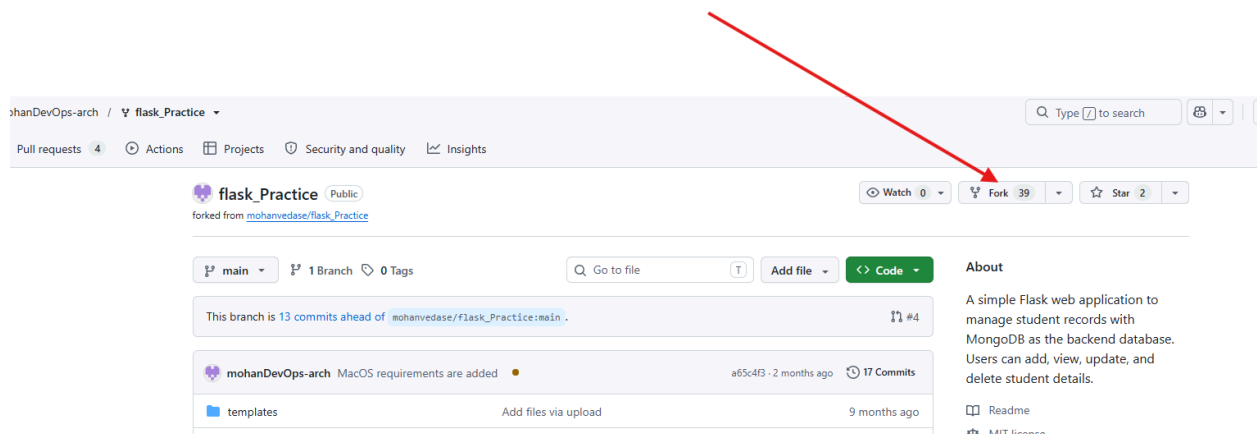
Purpose

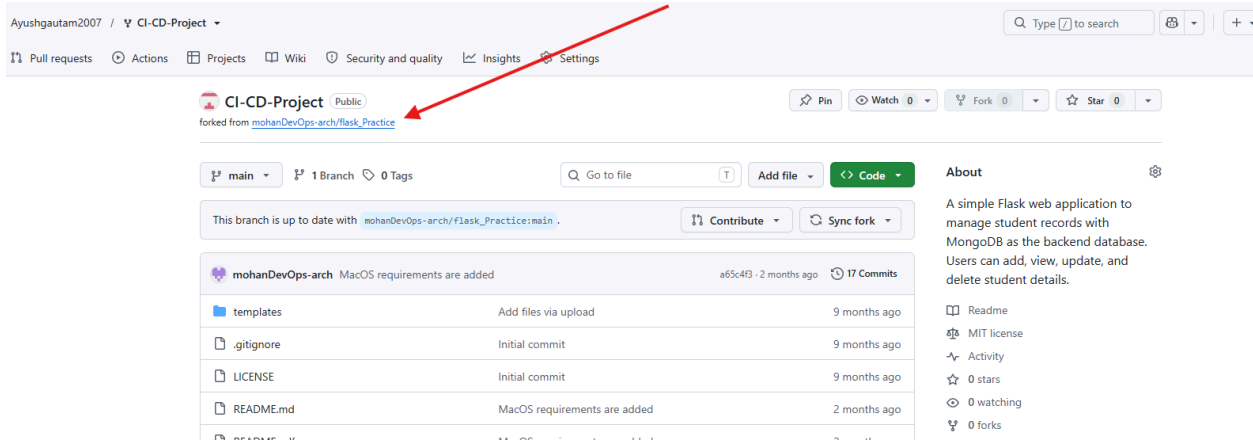
Python and pip are required to execute the Flask application and install all project dependencies.

2. Source Code:

The Flask application source code was forked from the original GitHub repository into my GitHub account. The forked repository was then cloned into the Jenkins server for pipeline execution.

1. Fork the github repository:





2. Clone the forked repository on Jenkins server:

```
[root@ip-172-31-0-147 /]# git clone https://github.com/Ayushgautam2007/CI-CD-Project.git
Cloning into 'CI-CD-Project'...
remote: Enumerating objects: 62, done.
remote: Counting objects: 100% (34/34), done.
remote: Compressing objects: 100% (18/18), done.
remote: Total 62 (delta 22), reused 16 (delta 16), pack-reused 28 (from 1)
Receiving objects: 100% (62/62), 150.42 KiB | 10.74 MiB/s, done.
Resolving deltas: 100% (27/27), done.
```

Purpose

Cloning the repository allows Jenkins to access the latest application source code and execute the pipeline.

3. Push Jenkinsfile on github repository:

The Jenkins pipeline is written using a Declarative Jenkinsfile. It consists of three major stages:

- Build
- Test
- Deploy

Each stage executes automatically whenever new code is pushed to the GitHub repository.

```
[root@ip-172-31-0-147 CI-CD-Project]# git status
On branch main
Your branch is up to date with 'origin/main'.

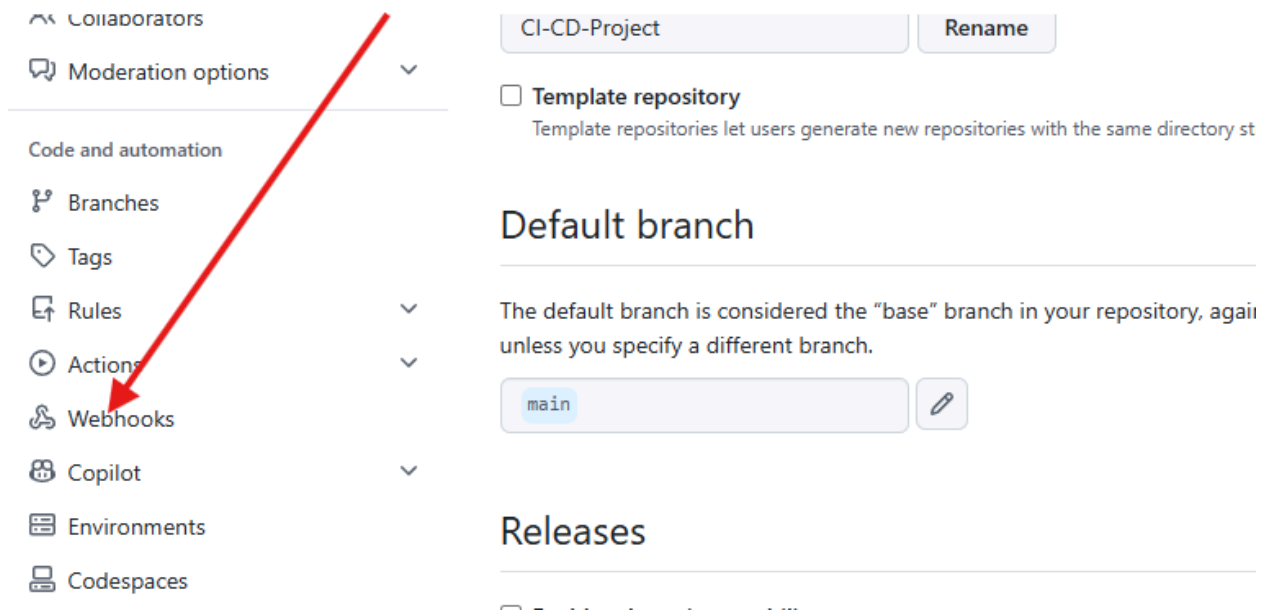
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Jenkinsfile

nothing added to commit but untracked files present (use "git add" to track)
[root@ip-172-31-0-147 CI-CD-Project]# git add .
[root@ip-172-31-0-147 CI-CD-Project]# git commit -m " Jenkinsfile for flask app"
[main c069308] Jenkinsfile for flask app
 1 file changed, 45 insertions(+)
 create mode 100644 Jenkinsfile
[root@ip-172-31-0-147 CI-CD-Project]# git push -u origin main
Username for 'https://github.com': Ayushgautam2007
Password for 'https://Ayushgautam2007@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 581 bytes | 581.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Ayushgautam2007/CI-CD-Project.git
   a65c4f3..c069308  main -> main
branch 'main' set up to track 'origin/main'.
[root@ip-172-31-0-147 CI-CD-Project]#
```

Purpose

The Jenkinsfile defines the CI/CD workflow and enables Jenkins to automate the complete build process.

4: Configure the pipeline to trigger a new build using webhooks:



Webhooks / Manage webhook

Settings Recent Deliveries

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

Content type *

Secret

Settings

Recent Deliveries

✓

5b6d11fe-737f-11f1-88a4-1bc60974131a

ping

redelivery

2026-06-29 11:30:34 ...

Request

Response

200

Redeliver

⌚ Completed in 1.24 seconds.

Headers

Request URL: http://3.110.55.235:8080/github-webhook/

Request method: POST

Accept: */*

Content-Type: application/json

User-Agent: GitHub-Hookshot/7621ddc

X-Github-Delivery: 5b6d11fe-737f-11f1-88a4-1bc60974131a

X-Github-Event: ping

X-Github-Hook-Id: 647523076

X-Github-Hook-Installation-Target-Id: 1283618940

X-Github-Hook-Installation-Target-Type: repository

Webhook Configuration

Payload URL:

http://<Jenkins-IP>:8080/github-webhook/

Content-Type:

application/json

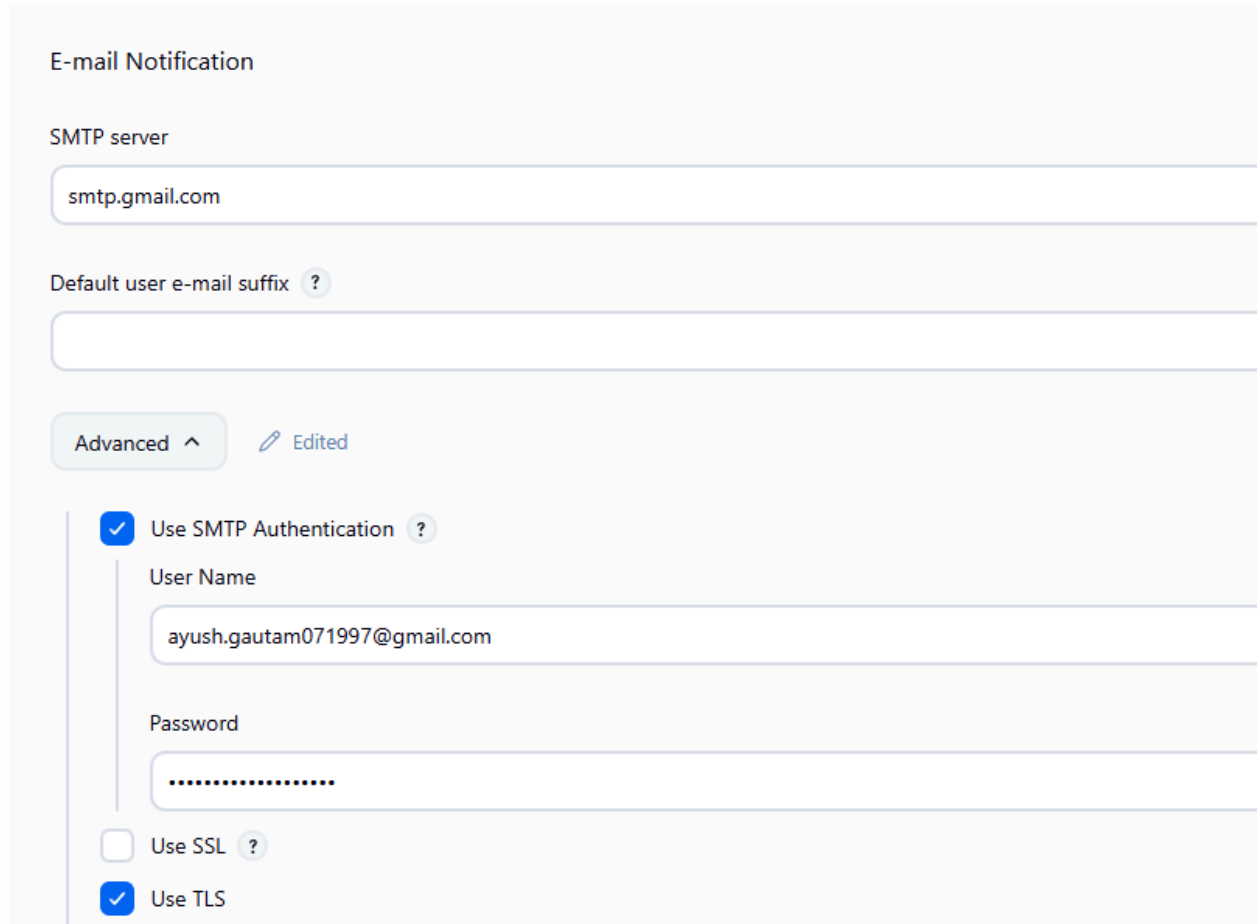
Trigger:

Push Events

Purpose:

Whenever new code is pushed to GitHub, Jenkins automatically starts a new build.

5. Set up a notification system to alert via email through Jenkins:



The screenshot shows the 'E-mail Notification' configuration page in Jenkins. The 'SMTP server' field is set to 'smtp.gmail.com'. The 'Default user e-mail suffix' field is empty. The 'Advanced' section is expanded, showing 'Use SMTP Authentication' checked, 'User Name' set to 'ayush.gautam071997@gmail.com', and 'Password' masked with dots. 'Use SSL' is unchecked, and 'Use TLS' is checked. An 'Edited' link is visible next to the 'Advanced' section header.

E-mail Notification

SMTP server

smtp.gmail.com

Default user e-mail suffix ?

Advanced ^ Edited

☒ Use SMTP Authentication ?

User Name

ayush.gautam071997@gmail.com

Password

.....

☐ Use SSL ?

☒ Use TLS

SMTP Server:
smtp.gmail.com

Port:
587

Authentication:
Enabled

TLS:
Enabled

Purpose:
Email notifications are sent automatically whenever the pipeline succeeds or fails.

6. screenshots of the Jenkins pipeline showing the build, test, and deployment stages:

1. Build Stage:

```
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] sh
+ python3 -m venv venv
+ . venv/bin/activate
++ deactivate nondestructive
++ '[' -n '' ']'
++ '[' -n '' ']'
++ hash -r
++ '[' -n '' ']'
++ unset VIRTUAL_ENV
++ unset VIRTUAL_ENV_PROMPT
++ '[' '!' nondestructive = nondestructive ']'
++ case "$(uname)" in
+++ uname
++ export 'VIRTUAL_ENV=/var/lib/jenkins/workspace/CI-CD Project/venv'
```

Purpose: The Build stage installs all Python dependencies required by the application using pip.

2. Test Stage:

```
Running unit tests...
[Pipeline] sh
+ pytest -v
===== test session starts =====
platform linux -- Python 3.9.25, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /var/lib/jenkins/workspace/CI-CD Project
collecting ... collected 4 items

test_app.py::test_home_page PASSED [ 25%]
test_app.py::test_add_student PASSED [ 50%]
test_app.py::test_update_student PASSED [ 75%]
test_app.py::test_delete_student PASSED [100%]
```

Purpose : The Test stage executes all unit tests using pytest to verify application functionality before deployment.

3. Deployment stage:

Purpose

The deployment stage starts the Flask application using the deployment script after successful execution of all test cases.

```
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
===== Deploy Stage =====
[Pipeline] sh
+ chmod +x start_flask.sh
+ ./start_flask.sh
===== Deploying Flask Application =====
===== Deployment Successful =====
[Pipeline] }
[Pipeline] // stage
```

Note

The complete Jenkins console output is available in the GitHub repository for reference.

Pipeline Execution Flow:

GitHub Push



Webhook Trigger



Jenkins



Checkout Source Code



Install Dependencies



Run Unit Tests

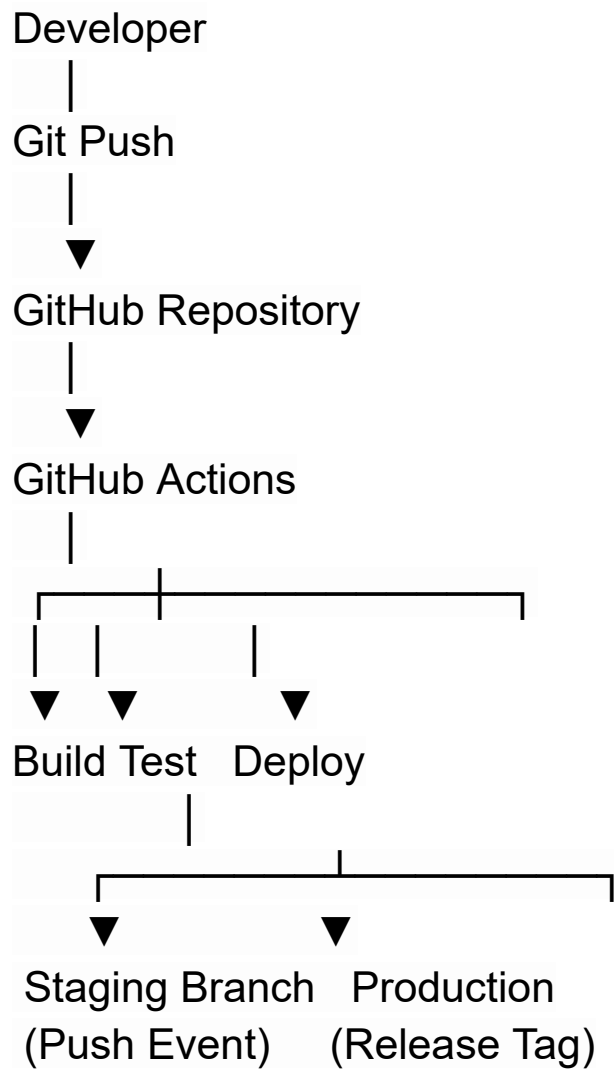


Deploy Application



Send Email Notification

GitHub Actions CI/CD Architecture:



2. GitHub Actions CI/CD Pipeline for Flask Application:

GitHub Actions is used to automate Continuous Integration and Continuous Deployment without requiring a dedicated Jenkins server.

The workflow automatically performs:

- Checkout Code
- Install Python
- Install Dependencies
- Run Tests
- Build Application
- Deploy to Staging
- Deploy to Production

This section demonstrates the implementation of GitHub Actions as a cloud-native CI/CD solution. The workflow automatically builds, tests, and deploys the Flask application based on GitHub events and release tags.

1. Ensure the repository has a main branch and a staging branch:

```
[root@ip-172-31-0-147 CI-CD-Project]# git branch -a
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
[root@ip-172-31-0-147 CI-CD-Project]# git checkout -b staging
Switched to a new branch 'staging'
[root@ip-172-31-0-147 CI-CD-Project]# git branch -a
  main
* staging
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
[root@ip-172-31-0-147 CI-CD-Project]# git push -u origin staging
Username for 'https://github.com': Ayushgautam2007
Password for 'https://Ayushgautam2007@github.com':
```

```
[root@ip-172-31-0-147 CI-CD-Project]# git branch -a
main
* staging
remotes/origin/HEAD -> origin/main
remotes/origin/main
remotes/origin/staging
[root@ip-172-31-0-147 CI-CD-Project]#
```

2. GitHub Actions Workflow:

```
[root@ip-172-31-0-147 CI-CD-Project]# mkdir -p .github/workflows
[root@ip-172-31-0-147 CI-CD-Project]# vim .github/workflows/ci-cd.yml
[root@ip-172-31-0-147 CI-CD-Project]#
```

3. Some Workflow Steps: complete yml file is uploaded on GitHub.

```
[root@ip-172-31-0-147 CI-CD-Project]# cat .github/workflows/ci-cd.yml
name: Flask CI/CD Pipeline

on:
  push:
    branches:
      - main
      - staging

  release:
    types:
      - published

jobs:
  build-test:
    name: Build and Test
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4
```

4. GitHub Secrets to store sensitive information required for deployments:

Code and automation

- Branches
- Tags
- Rules
- Actions
- Webhooks
- Copilot
- Environments
- Codespaces
- Pages

Security and quality

- Advanced Security
- Deploy keys
- Secrets and variables**
- Actions
- Agents
- Codespaces
- Dependabot

Integrations

- GitHub Apps
- Email notifications

Secrets Variables

Environment secrets

This environment has no secrets.

Manage environment secrets

Repository secrets

New repository secret

Name	Last updated
API_TOKEN	2 minutes ago
DEPLOY_KEY	2 minutes ago
PROD_API_TOKEN	now
PROD_DEPLOY_KEY	now
PROD_HOST	1 minute ago
PROD_USER	1 minute ago
STAGING_HOST	20 minutes ago
STAGING_USER	19 minutes ago

Repository Secrets

Secret	Purpose
STAGING_HOST	Staging Server IP
STAGING_USER	SSH Username
DEPLOY_KEY	SSH Private Key
API_TOKEN	API Authentication
PROD_HOST	Production Server
PROD_DEPLOY_KEY	Production SSH Key

5. GitHub Actions Build, Test, and Deployment:

Workflow Trigger

Main Branch

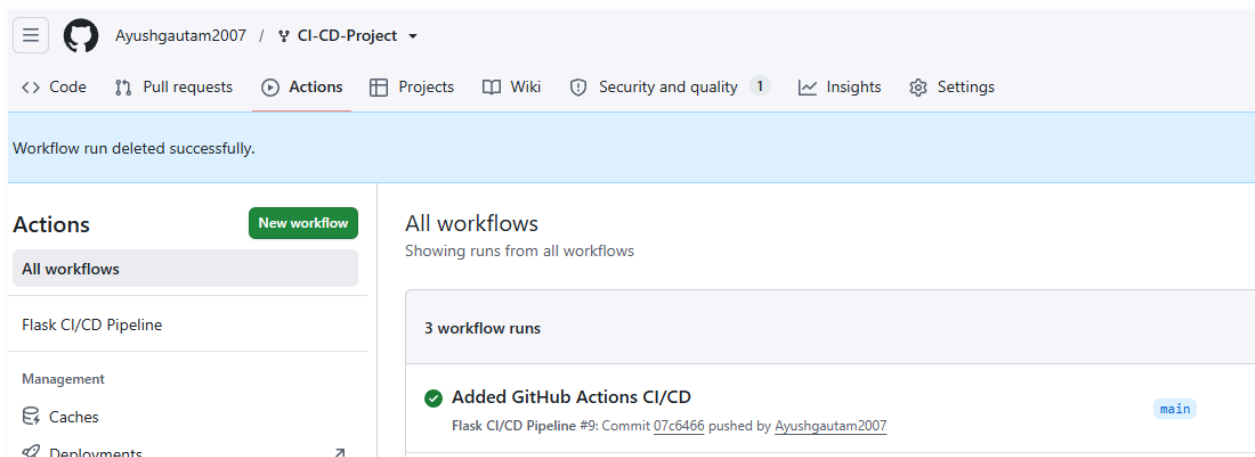
→ Build and Test

Staging Branch

→ Build, Test and Deploy to Staging

Release Tag

→ Deploy to Production



Challenges Faced:

1. Jenkins initially failed to trigger automatically due to webhook configuration issues.
2. Python dependencies were missing during the build stage.
3. Email notifications required Gmail SMTP authentication.
4. Deployment required executable permissions on the deployment script.
5. GitHub Secrets were configured to securely store deployment credentials.

Learning Outcomes

- Learned Jenkins Declarative Pipelines
- Configured GitHub Webhooks
- Implemented Continuous Integration
- Automated Flask application deployment
- Executed unit testing using pytest
- Configured GitHub Actions workflows
- Managed GitHub Secrets securely
- Configured Email Notifications in Jenkins

Conclusion

This project successfully demonstrates the implementation of end-to-end Continuous Integration and Continuous Deployment (CI/CD) for a Flask application using both Jenkins and GitHub Actions. Automated build, testing, deployment, webhook integration, GitHub Secrets, and email notifications reduce manual effort while improving software quality, reliability, and deployment efficiency.